DBMON-01: Database Monitoring Fundamentals

> **Series:** DBMON — Database Monitoring | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

This notebook introduces how Dynatrace monitors databases across your environment. You will learn how OneAgent automatically detects database calls through distributed tracing, how database spans capture query-level detail, and how the Dynatrace entity model represents database services. We also cover ActiveGate extensions for remote database monitoring and the full range of supported database technologies.

Table of Contents

1. How Database Monitoring Works
2. Database Span Anatomy
3. Discovering Database Services
4. Database Span Exploration
5. Database Types and Technologies
6. ActiveGate Extensions for Remote Monitoring
7. Baseline Database Metrics
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there)
OneAgent	Deployed on application hosts making database calls
Permissions	`storage:spans:read`, `storage:entities:read`, `storage:metrics:read`
Data	At least 1 hour of application traffic generating database calls

1. How Database Monitoring Works

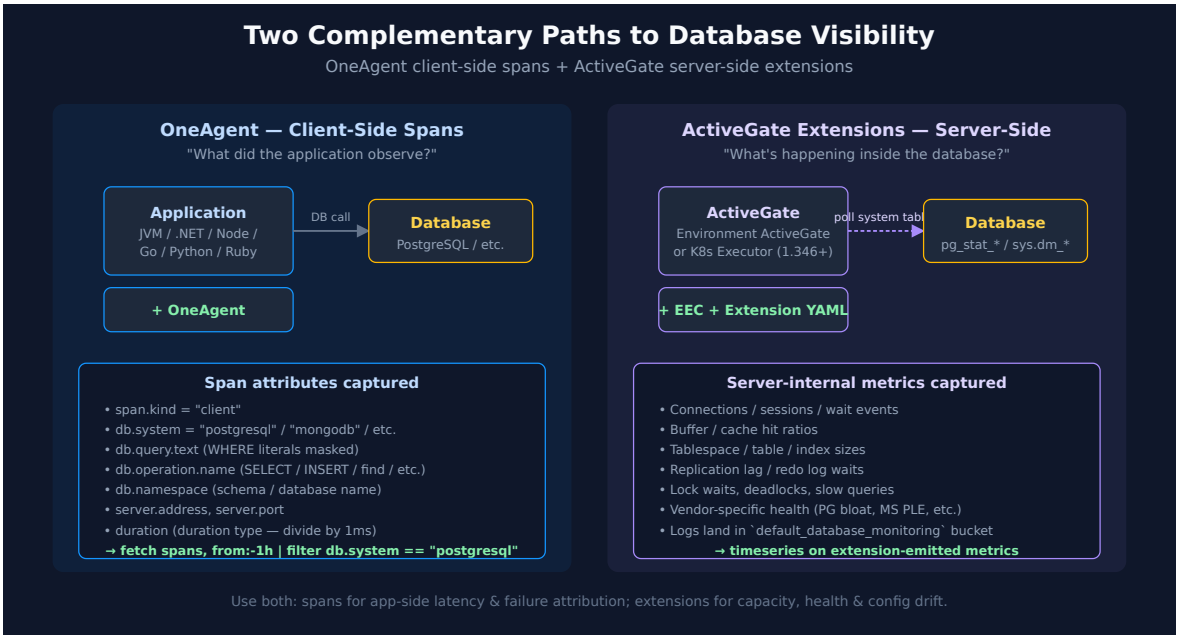
Dynatrace monitors databases through two complementary approaches:

Approach	How It Works	What It Captures
OneAgent Code-Level Instrumentation	Automatically instruments database client libraries (JDBC, ADO.NET, database drivers)	Query text, execution time, result counts, connection details
ActiveGate Extensions	Connects remotely to database management interfaces (SQL queries against system tables)	Internal DB metrics: tablespace usage, lock waits, buffer cache hit ratio

OneAgent captures every database call as a **span** in the distributed trace. These spans contain:

- **`db.system`** — The database technology (e.g., `postgresql`, `mysql`, `mongodb`)
- **`db.query.text`** — The query or command executed. Literals in a SQL `WHERE` clause are masked; parameterized statements keep their own placeholders (`?`, `@p0`)
- **`db.operation.name`** — The operation type (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `find`, `aggregate`)
- **`db.namespace`** — The database or schema name
- **`server.address`** — The database server hostname or IP
- **`server.port`** — The database server port

> **Note:** Dynatrace masks literals in a SQL `WHERE` clause — *"Literals that are part of the WHERE clause of an SQL statement are replaced with ****". Parameterized statements (prepared statements, ORMs) already carry placeholders such as `?` or `@p0`, so they group by `db.query.text` cleanly; SQL built with inline values outside the `WHERE` clause (for example `INSERT … VALUES (1, 'x')`) does not.
>
> **Sources:** Personal data captured by Dynatrace (DT docs) — *"Literals that are part of the WHERE clause of an SQL statement are replaced with ***** , for example, WHERE userId = '*******' ."*



2. Database Span Anatomy

Let's examine the structure of a database span by querying for recent database calls and inspecting the available fields.

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Inspect database span fields — sample 10 recent DB calls
fetch spans, from:-1h
| filter isNotNull(db.system)
| fields start_time, db.system, db.operation.name, db.namespace,
        db.query.text, server.address, server.port,
        duration, span.kind, dt.service.name
| sort start_time desc
| limit 10
```

The query above returns the key attributes of each database span:

Field	Description	Example
`db.system`	Database technology identifier	`postgresql`, `mysql`, `mongodb`
`db.operation.name`	SQL command or DB operation	`SELECT`, `INSERT`, `find`
`db.namespace`	Database or schema name	`orders_db`, `inventory`
`db.query.text`	Normalized query text	`SELECT * FROM orders WHERE id = ?`
`server.address`	Database host	`db-prod-01.internal`

Field	Description	Example
`duration`	Execution time — a `duration` value; divide by `1ms` for milliseconds	`1.5 ms`
`span.kind`	`client` for outgoing DB calls (lowercase)	`client`

`db.system` is `experimental` in the semantic dictionary and `db.query.text` is not always present — on the validation tenant (10/02/2026) it was set on 58% of SQL Server spans and on no Redis spans.

3. Discovering Database Services

Dynatrace records each database call as a **client span on the calling service**, described by `db.*` attributes — the service, not a separate database entity, is where the impact shows. Let's discover which services call which databases.

> **Sources:** Database query monitoring in Services (DT docs) — *"When a service makes a database call, Dynatrace captures it as a database client span."*

```
// Discover which services call databases, and what they call.
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Note: `serviceType`, `databaseVendor`, `databaseHostNames` and
`softwareTechnologies` are
// CLASSIC dt.entity.service attributes — the Smartscape SERVICE node does not
carry them
// (its model is id / id_classic / name / type / tags / lifetime /
references), so filtering
// a Smartscape node on serviceType == "DATABASE_SERVICE" matches nothing.
Under service
// detection v2 there is no separate DATABASE_SERVICE entity at all: a
database call is a
// client span on the calling service, described by db.* attributes. Start
from the spans
// and resolve the service identity from there.
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize call_count = count(), by:{calling_service = dt.service.name,
db.system, server.address}
| sort call_count desc
| limit 50
```

You can also discover databases through the spans themselves, which is useful when entity detection hasn't yet completed or when you want to see databases called from specific services.

```
// Discover database technologies from span data
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    distinct_statements = countDistinct(db.query.text)
}, by:{db.system, db.namespace, server.address}
| sort call_count desc
| limit 20
```

4. Database Span Exploration

Understanding the distribution of database calls helps identify which databases are most heavily used and where optimization efforts should focus.

> **Calls are not statements.** A database span is one *call* from the client. On SQL spans, many calls are not statements: `CONNECT` opens a connection and `COMMIT` ends a transaction (neither carries `db.query.text`), while `PREPARE` and `RESULTSET` are driver phases that repeat the text of a statement span in the same trace. On the validation tenant (SQL Server, 1 h, 10/02/2026) those four operations were 19,379 of 42,314 calls, `CONNECT` alone 16,723 at about 1 ms each. That inflates a query count by more than 80% and pulls every latency percentile down. Cells in this series that measure queries or query latency drop those four operations with:
>
> `dql > | filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT", "PREPARE", "RESULTSET"})) >`
>
> `coalesce` keeps spans with no `db.operation.name` (MongoDB and Redis on the validation tenant). Without it, `in()` returns null for them and the filter drops them. Cells that count calls or failures keep every span.

```
// Database call volume by technology over the last hour
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    total_calls = count(),
    avg_duration_ms = avg(duration) / 1ms,
    p95_duration_ms = percentile(duration, 95) / 1ms,
    max_duration_ms = max(duration) / 1ms
}, by:{db.system}
| sort total_calls desc
```

```
// Database call volume over time — identify traffic patterns
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries call_count = count(), by:{db.system}, interval:5m
```

```
// Distribution of database operations (SELECT vs INSERT vs UPDATE vs DELETE)
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.operation.name)
| summarize op_count = count(), by:{db.system, db.operation.name}
| sort db.system asc, op_count desc
```

5. Database Types and Technologies

Dynatrace supports a broad range of database technologies through OneAgent auto-instrumentation.

> **Coverage is per client library, not per database.** "Auto-instrumented" describes the *driver* the application uses, so support arrives library by library and OneAgent version by OneAgent version. Redis on .NET is the clearest example: **StackExchange.Redis** is auto-instrumented from **OneAgent 1.337** and **ServiceStack.Redis** only from **OneAgent 1.343** — the same database, two client libraries, six sprints apart. Where a `db.system` value below is missing from your spans, check the client library and the OneAgent version on the calling host before concluding the database is unmonitored.

Category	Technologies	`db.system` Values
Relational (SQL)	PostgreSQL, MySQL, MariaDB, Microsoft SQL Server, Oracle, IBM Db2	`postgresql`, `mysql`, `mariadb`, `mssql`, `oracle`, `db2`
NoSQL Document	MongoDB, Couchbase, Amazon DynamoDB, Azure Cosmos DB	`mongodb`, `couchbase`, `dynamodb`, `cosmosdb`
NoSQL Key-Value	Redis, Memcached, Amazon ElastiCache	`redis`, `memcached`
NoSQL Column	Apache Cassandra, Apache HBase, ScyllaDB	`cassandra`, `hbase`
Search Engines	Elasticsearch, OpenSearch, Apache Solr	`elasticsearch`, `opensearch` (Solr: no dictionary value)
Message Brokers	Apache Kafka, RabbitMQ, Amazon SQS	not `db.system` — brokers use **`messaging.system`** (`kafka`, `rabbitmq`, `aws_sqs`); see DBMON-04
Graph	Neo4j, Amazon Neptune	`neo4j`, `neptune`

Important: The `db.system` field follows the OpenTelemetry semantic conventions. The exact values may vary depending on the database driver and instrumentation version.

Semantic Dictionary 1.340 (May 2026) — first-class Smartscape database models. The Semantic Dictionary now defines dedicated Smartscape models for major database technologies: Oracle Database (ASM disk group, cluster, instance, database), SQL Server (availability database / group / replica, instance, database), SAP HANA (database, instance, service), IBM Db2 (database member, instance, tablespace), MySQL, MariaDB, and PostgreSQL (database + instance each) — **21 models** in total, each with `belongs_to` / `runs_on` / `is_part_of` / `same_as` / `uses` relationship properties. (Verified against the live Semantic Dictionary: `fetch dt.semantic_dictionary.models | filter startsWith(name, "dt.smartscape.db_")` returns 21.) As these roll out, expect database topology to surface as typed Smartscape nodes alongside the span-based view in §3. The span-level `db.system` analysis in this series is unaffected.

OneAgent 1.339 (June 2026) — Db2 naming is now mixed case. Db2 database naming switched to mixed case, so saved DQL or filters that match Db2 *entity or database names* case-sensitively may need updating. The `db.system == "db2"` span-attribute filters used throughout this series follow the OpenTelemetry semantic convention (lowercase value) and are unaffected.

Sources: Technology support (DT docs) — .NET *Database framework* table: StackExchange.Redis *"2.0.0+"*, ServiceStack.Redis *"5.0.0+"*, What's new in OneAgent 1.337 (DT docs) — StackExchange.Redis auto-instrumentation, What's new in OneAgent 1.343 (DT docs) — ServiceStack.Redis auto-instrumentation. **Derived:** the per-client-library-not-per-database framing generalizes the Redis pair; no single page states it as a rule. **Dictionary:** `db.system` (`experimental`) — `supported_values` include `mysql` and `mariadb` as separate values, and no `solr` , `kafka` or `rabbitmq` , read 10/23/2026.

```
// Discover which database technologies are active in your environment
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    unique_databases = countDistinct(db.namespace),
    unique_servers = countDistinct(server.address)
}, by:{db.system}
| sort call_count desc
```

6. ActiveGate Extensions for Remote Monitoring

While OneAgent captures database calls from the application side (client spans), ActiveGate extensions monitor the database server itself. This provides internal metrics that are invisible from the application perspective.

> **Recommendation for new customers:** build on **Extensions 2.0** — the current extensions framework. Extensions Framework 1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31). JMX and PMI EF1.0 extensions were the exception, supported past March 2025 as deprecated; they now have their own end-of-support date of **July 1, 2027** for SaaS environments. Note that recent Dynatrace doc pages say simply "Extensions" and mean Extensions 2.0.

Extension Architecture

Component	Role
ActiveGate	Hosts the extension runtime. Two deployments: an **Environment ActiveGate** (the classic path), or — for SQL extensions, from Dynatrace 1.346 with Dynatrace Operator 1.8+ — **SQL Extension Executor pods in Kubernetes**, coordinated by the EEC and forwarding through an ActiveGate pod. Dynatrace 1.346 is rolling out to tenants in stages — verify it has reached yours; until then the Environment ActiveGate is the working path.

Component	Role
Extension Package	YAML-defined declarative package executed by the Extension Execution Controller (EEC). Most database extensions ship with built-in SQL/Prometheus/SNMP data sources; custom logic can be added via an optional Python data source
Monitoring Configuration	Defines connection string, credentials, polling interval, and the destination Grail bucket
`default_database_monitoring` bucket	Where extension logs land by default; reference this bucket in IAM policies for DB-team access scoping (see ORGNZ-02 + IAM-04/05)

Available Extensions

Extension	Metrics Captured
PostgreSQL	Connections, transactions/sec, tuple operations, table/index sizes, lock waits, replication lag
MySQL	Connections, queries/sec, buffer pool hit ratio, InnoDB row operations, slow queries
MS SQL Server	Batch requests/sec, page life expectancy, buffer cache hit ratio, deadlocks, wait stats
Oracle	Sessions, tablespace usage, SGA/PGA memory, redo log waits, library cache hit ratio
MongoDB	Connections, operations/sec, document metrics, replica set health, storage engine stats

> **Note:** the SQL / Prometheus / SNMP data sources these database extensions use run on an **Environment (host-based) ActiveGate**, which remains the working path. **SQL** extensions now have a second option: *Run SQL extensions on Kubernetes* documents Oracle, Microsoft SQL Server, PostgreSQL, MySQL / MariaDB, SAP HANA, IBM Db2 and Generic JDBC running on SQL Extension Executor pods that Dynatrace Operator 1.8+ deploys, with Dynatrace 1.346+ and SQL Extension Executor / EEC 1.345+ as prerequisites. Dynatrace 1.346 reaches tenants through a

> staged rollout — confirm your tenant version before choosing it. Both pages are cited below; Prometheus and SNMP data sources are not covered by them.

Where to Go Deeper

- **AUTOM series** (14 notebooks) — GitOps / Monaco / Terraform automation for extension deployment at scale
- **ORGNZ-02** — Bucket strategy (including `default_database_monitoring`)
- **IAM-04 / IAM-05** — Policy design that scopes extension log access by bucket

> **Sources:** Extensions (DT docs) — the Extensions 2.0 framework and the EEC execution model; note recent pages say simply *"Extensions"* and mean 2.0, Extension data sources (DT docs) — the built-in SQL / Prometheus / SNMP data sources and the optional Python data source, Run SQL extensions on Kubernetes (DT docs) — *"The following database types are supported, both as official Dynatrace Hub extensions and as custom extensions"*; prerequisites Dynatrace 1.346+, SQL Extension Executor 1.345+, Extension Execution Controller 1.345+, Enable Dynatrace SQL database extensions (DT docs) — *"Executor pods query your databases and collect metrics. The EEC distributes tasks across executor pods and forwards data to the ActiveGate, which enriches and routes it to the Dynatrace backend."*, End-of-support announcements (DT docs) — lists *"Dynatrace Extension Framework 1.0"* under the 2025-03-31 end-of-support date and *"Dynatrace Extension Framework 1.0 (Python 3.8)"* under 2024-10-31, with the note *"Note that JMX and PMI Extensions Framework 1.0 are supported past March 2025 but are deprecated"*, re-read 09/28/2026, EF1 JMX and PMI extensions end of support (DT docs) — *"As of July 1, 2027, all Extension Framework 1.0 JMX and PMI extensions will be out of support for SaaS Environments"*.
>
> **New (SaaS 1.346 — staged rollout from 08/25/2026): a `definity` extension for data-pipeline observability.** Verbatim: *"The new definity extension is introduced. It ingests pipeline health, performance, and cost data into Dynatrace, giving you visibility into your lakehouse and Spark pipeline ecosystem."*
>
> This sits **adjacent to** the database monitoring in this series rather than inside it. The extensions above monitor a *database* — connections, queries, locks, replication. Definity monitors the *pipelines that move data into and around a lakehouse*, and adds **cost** as a first-class signal, which none of the database extensions do. If your estate runs Spark or a lakehouse alongside the databases covered here, it fills a gap this series does not otherwise address; if it does not, there is nothing to do. Check the Hub for availability on your tenant.

> **Sources:** What's new in Dynatrace SaaS 1.346 (DT docs) — the definity extension entry quoted above, read 08/28/2026.

Where Extension Logs Land in Grail

Monitoring logs generated by the official Dynatrace database extensions are stored in a dedicated Grail bucket: `default_database_monitoring` (introduced in Dynatrace SaaS 1.337, through a default bucket-assignment rule that is applied first and cannot be overwritten). The Databases app itself now filters via this bucket for performance; filtering your own queries on the bucket narrows the scan the same way, and lets you scope IAM policies tightly to database-monitoring data.

```
fetch logs, from:-1h
| filter dt.system.bucket == "default_database_monitoring"
| filter dt.extension.name == "com.dynatrace.extension.postgres"
| limit 50
```

Grant access to this bucket with a bucket-scoped policy — both the bucket permission and the `logs` table permission are needed:

```
ALLOW storage:buckets:read WHERE storage:bucket-name =
"default_database_monitoring";
ALLOW storage:logs:read WHERE storage:bucket-name =
"default_database_monitoring";
```

> **Sources:** What's new in Dynatrace SaaS 1.337 (DT docs) — introduces the `default_database_monitoring` bucket, Assign permissions in Grail (DT docs) — *"All bucket permissions need to start with storage:buckets:read"* and the table-permission form *"ALLOW storage:logs:read WHERE storage:bucket-name="default_logs";"*. The extension IDs on this bucket (including `com.dynatrace.extension.postgres`) are listed in the SaaS 1.337 entry above, alongside *"The new default rule is always applied first and cannot be overwritten"*.

Dynatrace Database App — Analysis-First Observability

Beyond OneAgent spans and ActiveGate extension metrics, Dynatrace ships a dedicated **Databases app** that layers automated analysis on top of both data sources. Announced 08/04/2026 as available for PostgreSQL and MySQL; the docs page (updated 08/05/2026)

now lists monitored vendors *"such as PostgreSQL, MySQL, MSSQL, Oracle, MariaDB, and HanaDB"*. (Neither source applies a **GA** or preview label, so none is used here.) Verify the app is enabled on your tenant before relying on it.

The announcement describes five analysis areas — configuration, schema, query, execution plan, and a rolled-up **Health Score** — with AI-generated remediation. The health-score concept page documents the score: *"The health score is the weighted average of all checks"*, combined into *"a unified score from 0 to 100"*, and the checks are vendor-specific — *"Dynatrace health checks are vendor-specific, such as PostgreSQL, MySQL, and Microsoft SQL Server"*. The instance list separately shows health as alert state: *"Health highlights instances with critical, warning, or no active alerts"*.

Capability	What it adds beyond this notebook's DQL approach
Health Score	Single 0–100 rating per database instance for triage before deep analysis
Query & execution-plan insights	Interactive execution-plan visualization, normalized across PostgreSQL, MySQL, and SQL Server
Configuration insights	Flags parameter mismatches against recommended settings
Schema insights	Structural warning signals (missing indexes, oversized tables)
AI-powered remediation	Suggested fix with reasoning, generated by Dynatrace Intelligence

Treat the app as a triage front end, not a replacement for the span- and extension-level analysis in this series — the DQL patterns in DBMON-01/05/06 remain the way to build the custom dashboards, alerts, and SLOs the app's UI doesn't cover.

> **Sources:** Introducing Intelligent Database Observability (DT blog) — the announcement is the **only** source for the vendor list (PostgreSQL, MySQL), the five analysis areas and the AI-generated remediation; treat those as announcement-backed, not documentation-backed. Database health score overview (DT docs) — *"The health score is the weighted average of all checks."* Databases app (DT docs) — the vendor list *"such as PostgreSQL, MySQL, MSSQL, Oracle, MariaDB, and HanaDB"* and the alert-based Health column, re-read 10/23/2026.

7. Baseline Database Metrics

Establishing a performance baseline is essential for detecting anomalies. The following queries help you understand your typical database performance characteristics.

```
// Database response time baseline — hourly P50, P95, P99 over the last 24
hours
fetch spans, from:-24h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries {
    p50_ms = percentile(duration / 1ms, 50),
    p95_ms = percentile(duration / 1ms, 95),
    p99_ms = percentile(duration / 1ms, 99)
  }, interval:1h
```

```
// Error rate baseline — database call failures over time
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries {
    total = count(),
    errors = countIf(span.status_code == "error", default:0)
  }, interval:1h
| fieldsAdd error_rate_pct = round(arraySum(errors) / arraySum(total) * 100,
decimals:2)
```

8. Summary and Next Steps

In this notebook you learned:

- How Dynatrace captures database activity through OneAgent span instrumentation
- The key span attributes (`db.system`, `db.query.text`, `db.operation.name`, `db.namespace`) that describe each database call
- How to discover which services call which databases, and which technologies are active
- The role of ActiveGate extensions for server-side database metrics
- How to establish a performance baseline using span data

Next Steps

- **DBMON-02: SQL Database Monitoring** — Deep dive into relational database analysis with PostgreSQL, MySQL, MS SQL, and Oracle
- **DBMON-03: NoSQL Database Monitoring** — MongoDB, Cassandra, DynamoDB, and Cosmos DB analysis

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.